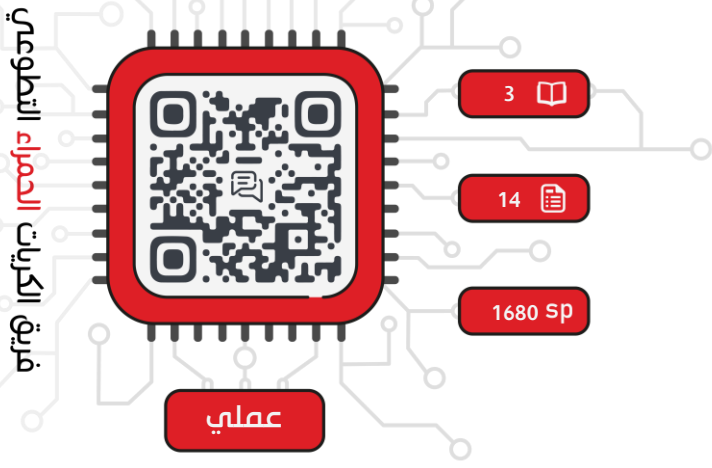


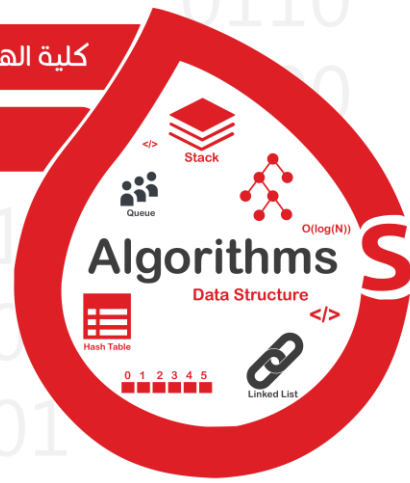


كلية الهندسة المعلوماتية

السنة الثانية

الـ LinkedList و تعقيد العودية

عملي مشترك



RB Informatcs; 22/11/2025

خوارزميات 1

سنحدث اليوم عن التعقيد الزمني للتتابع العودية :
و هو يتعلق بأمرين:

1. الخطوات الأساسية التي يتم تنفيذها في الخوارزمية (Basic Steps).

2. تكرار هذه الخطوات.

توضيح:

بالعودة إلى مثال العامل نجد:

```
1 int fact(int n) {  
2     if(n == 0 || n == 1)  
3         return 1;  
4     return n*fact(n-1);  
5 }
```

في هذه الخوارزمية يتم تكرار الكود n مرة وصولاً إلى 1
في كل مرة تتكرر الخطوة $(n-1)$

Time Complexity: $T(n) = 3 + T(n - 1)$

$$\begin{aligned} &= 3 + 3 + T(n - 2) \\ &= 3 + 3 + 3 + T(n - 3) \\ &\vdots \\ &= 3k + T(n - k) \end{aligned}$$

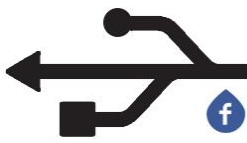
شرط التوقف:

$$n - k = 0 \rightarrow n = k$$

$$T(0) = 1$$

$$T(n) = 3n + T(0) = 3n + 1$$

$\Rightarrow$ Time Complexity $\approx O(n)$



و الآن لنحسب تعقيد الأكواد التالية:

الكود الأول:

```

1 int fun(int a, int b) {
2     if(b == 0)
3         return a;
4     return fun(a, b-1) + 1;
5 }

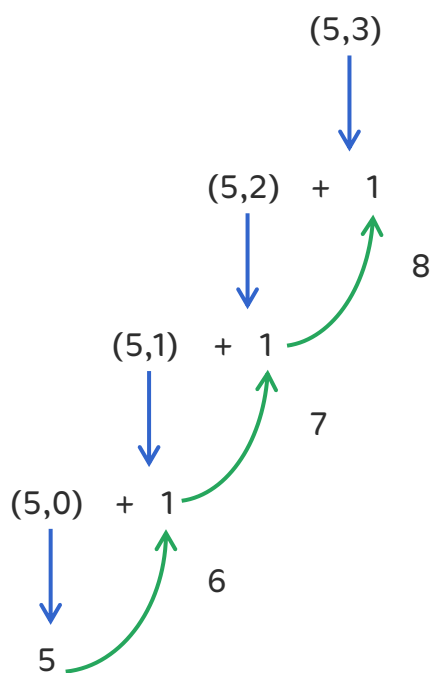
```

الكود يقوم بجمع رقمين و يتكرر حسب قيمة b

كل مرة b-1

إذاً فالتعقيد حسب المدخل b (أي $O(b)$).

آلية عمل الكود:



الكود الثاني:

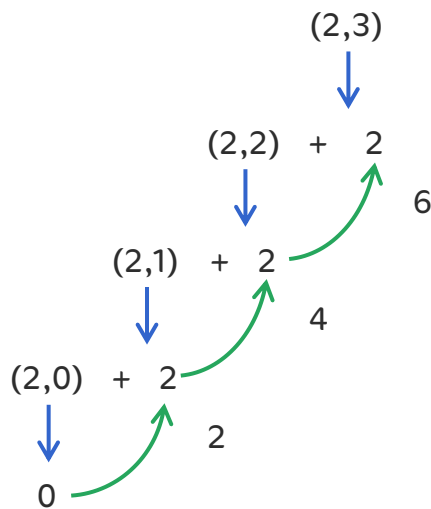
```

1 int fun(int a, int b) {
2     if(b == 0)
3         return 0;
4     return fun(a, b-1) + a;
5 }

```

الكود يفيد في إيجاد ضرب رقمين عن طريق جمع a بمقدار b مرة:

 $2 + 2 + 2 = 6 \Rightarrow 3$ مراتأيضاً حسب قيمة b يتم الاستدعاء $O(b)$.

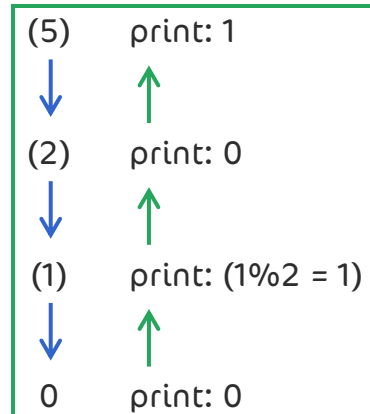


و أيضاً بتغيير بسيط على الكود السابق نحصل على خوارزمية لحساب power بنفس التعقيد السابق:

```
1 int fun(int a, int b) {
2     if (b == 0)
3         return 1;    // a^0 = 1
4     return fun(a, b-1) * a;
5 }
```

الكود الثالث: (وهو للتحويل من النظام العشري للثنائي)

```
1 int fun(int n) {
2     if (n > 0)
3         fun(n/2);    // a^0 = 1
4     System.out.println(n/2);
5 }
```



الشرح:

يتم تحويل 5 $\leftarrow$ 0101

في كل مرة طالما $n > 0$ سيستدعي التابع من جديد دون أن يكمل الخطوات حتى يصل إلى $n = 0$ عندها يعود ليكمل التعليمات بعد استدعاء fun بشكل تصاعدي.

في كل مرة الخطوة (step) هي $n/2$ إذاً التعقيد هو $O(\log n)$.

و هلاً صار دورك لتفكر بتعقيد هالخوارزمية (فيبوناتشي):

```
1 int fun(int n) {
2     if(n == 0)
3         return 0;
4     else if(n == 1)
5         return 1;
6     return fun(n - 1) + fun(n - 2);
7 }
```

قبل البدء بال LinkedList...

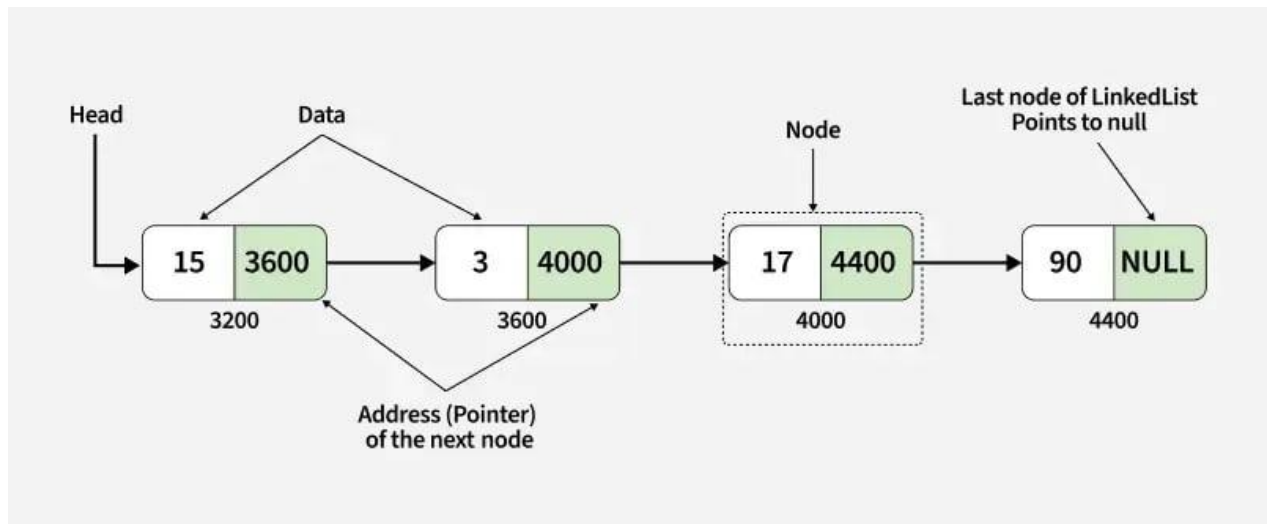
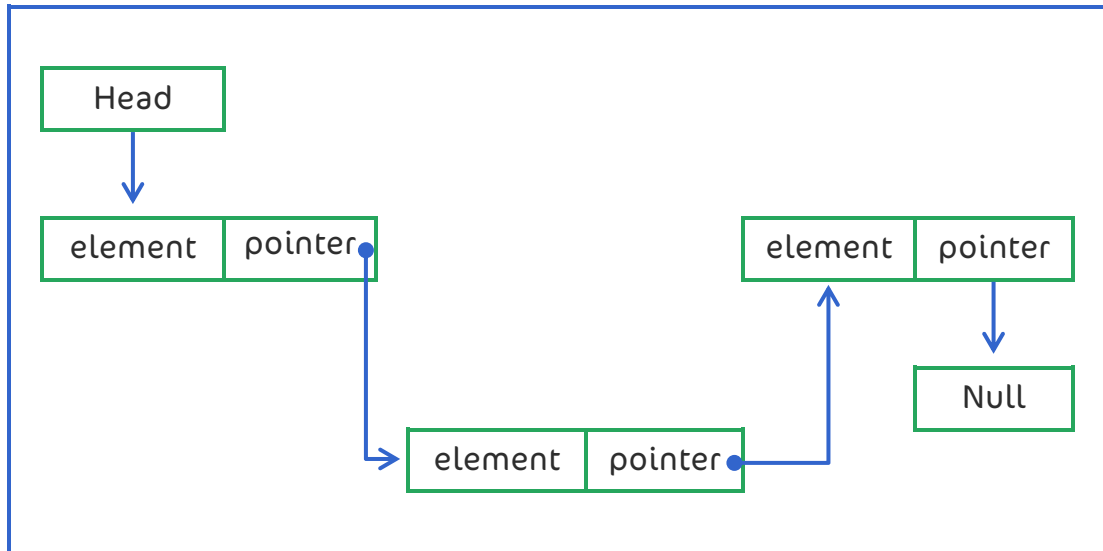
سنستذكر ال Array

- الوصول إلى أي عنصر يكون عن طريق ال index بتعقيد $O(1)$.
- إضافة عنصر إلى مصفوفة (ضمن نطاق حجم المصفوفة):
الإضافة فقط على الأخير، وإذا أردنا إضافة عنصر في اندكس معين سيكون عن طريق إزاحة جميع العناصر بعد هذا ال index إلى اليمين ثم إضافة العنصر.
- وكذلك عند حذف عنصر:
نحذف العنصر من ال index المطلوب ثم نزيح العناصر إلى اليسار.
- Fixed Size:
حجم المصفوفة ثابت، فمثلاً لو أردنا إضافة عنصر جديد و حجم المصفوفة لا يسمح بذلك سيكون علينا بناء مصفوفة جديدة و نقل جميع العناصر إليها و إضافة العنصر الجديد.
- sequential التسلسلية:
أي تتخزن عناصر المصفوفة في مواقع متجاورة في الذاكرة، و هذه مشكلة فقد لا تتوفر المساحة المطلوبة للمصفوفة بشكل متسلسل، في هذه الحالة سنواجه مشكلة لأننا لن نتمكن من حجز المصفوفة.

ما هي ال LinkedList؟؟؟

هي بنية معطيات خطية تخزن العناصر فيها بأماكن متفرقة عشوائية في الذاكرة و ترتبط هذه العناصر مع بعضها من خلال مؤشرات (pointers).

تتكون ال LinkedList من عقد (nodes) وكل عقدة تحتوي على حقلين أحدهما يخزن بداخله عنصر (من نوع string, float,.... int) و الحقل الآخر يحتوي على مؤشر يشير للعقدة التالية و آخر عقدة تُوّشّر على null.



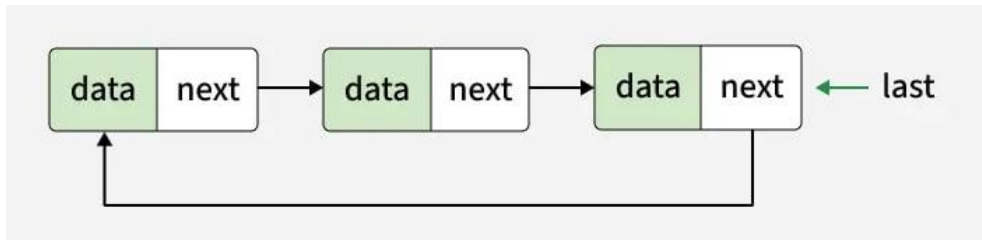
المؤشرات Pointers: متحولات تحوي عنوان متغير آخر في الذاكرة (address) و هذا مفيد في تقليل حجم الذاكرة المستخدمة.

head: هو مؤشر يشير إلى العقدة الأولى، و القيمة الخاصة به هو عنوان هذه العقدة.

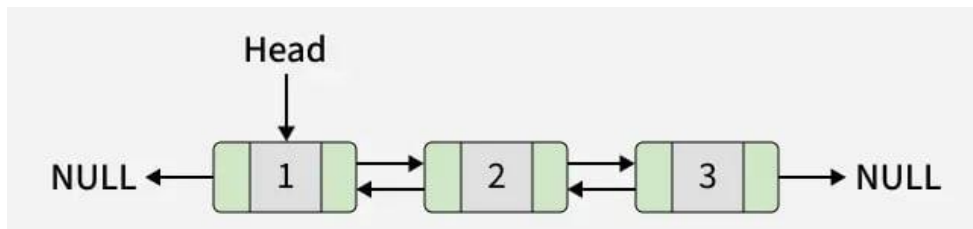
LinkedList	array
حجم ديناميكي.	حجم ثابت.
تحجز بشكل عشوائي في الذاكرة.	تحجز بشكل متتابع في الذاكرة.
الوصول لعنصر ما يتطلب المرور على جميع العناصر قبله.	الوصول عن طريق الـ index. $O(1)$
الحجز في الذاكرة أثناء الـ run time.	الحجز في الذاكرة أثناء الـ compile time.
عملية الإضافة و الحذف تتم بـ $O(1)$ حيث يمكن إضافة عناصر في بداية أو نهاية المصفوفة (back-front) أو في مكان معين.	عملية الإضافة و الحذف تتم بـ $O(n)$.

أنواع ال LinkedList

1. Single: كما تعرّفنا عليها في الأعلى.
2. Circular: آخر عنصر يشير إلى أول عنصر.



3. Double: مؤشرين لكل عقدة (prev للعنصر السابق و next للعنصر اللاحق).



كيفية استخدام LinkedList في جافا

إنشاء class Node:

```

1 public class Node {
2     int element;
3     Node next;
4     public Node(int element){
5         this.element = element;
6         this.next = null;
7     }
8 }

```

ملاحظة:

باني Node يستقبل فقط العنصر و يخزنه في العقدة و يؤشر ب null للعقدة التي تليه، لأننا نتعامل مع إضافة node في نهاية LinkedList (back).

إنشاء class LinkedList يضم عدة دوال سنقوم بتعريفه أولاً :

يحتوي class LinkedList على head الذي يعتبر الأساس و يشير إلى أول عقدة ، و في البداية يُؤشر إلى null إذا كانت الـ LinkedList فارغة.

```
1 class LinkedList{
2     Node head;
3 }
```

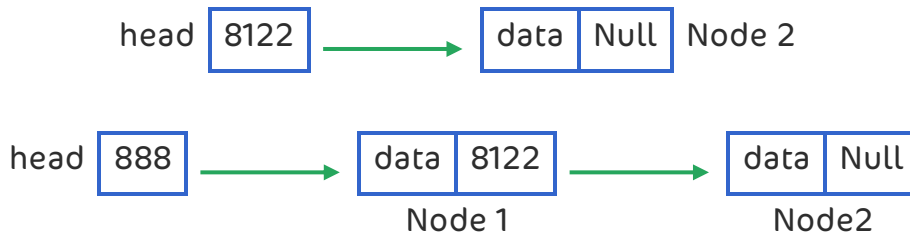
الدالة (1): إضافة عنصر في البداية (front):

```
1 public void insertFront(int data) {
2     Node newNode = new Node(data);
3     newNode.next = head;
4     head = newNode;
5 }
```

نجعل head يشير إلى العقدة الجديدة ، و العقدة الجديدة تشير إلى العقدة التي تليها (كما هو موضح في الرسم).

ملاحظة:

عندما نكتب `newNode.next=head` هذا يجعل العقدة الجديدة تشير إلى نفس المكان الذي يشير إليه head قبلاً، و عندما نكتب `head=newNode` هذا يجعل head بعد الإضافة يُؤشر إلى العقدة الجديدة.



■ التعقيد: $O(1)$.

فائدة أريسيزية:

لا تجعل head هو الذي يتحرك و يشير إلى عناصر الـ list لأن هذا سيجعل العناصر مبعثرة في الذاكرة و غير مربوطة ببعضها ، بل عرّف متحولاً `current` أو `temp_node` و استخدمها للمرور على عناصر الـ list (سيتم استخدام هذه الطريقة في الدوال القادمة).

الدالة (2): إضافة عنصر في نهاية LinkedList (Back):

```

1 public void insertBack(int data) {
2     Node newNode = new Node(data);
3     if (head == null) {
4         head = newNode;
5         return;
6     }
7     Node last = head;
8     while (last.next != null)
9         last = last.next;
10    last.next = newNode;
11    newNode.next = null;
12 }

```



الشرح:

- حلقة فيها if تتحقق فيما إذا كانت head=null عندئذ ستكون node المضافة هي الأولى أي أن الـ LinkedList فارغة.
- تعريف node اسمها last و الهدف منها إيجاد آخر عنصر في الـ LinkedList حيث سنمرر last من head حتى نجد العنصر الأخير و الذي يكون المؤشر فيه null، عندئذ يتم كسر حلقة while و تؤشر last على newNode و يأخذ مؤشر newNode القيمة null.

■ التعقيد: $O(n)$.

الدالة (3): إضافة عنصر في موضع معين:

```

1 public void insertAt(int index, int new_data) {
2     Node newNode = new Node(new_data);
3     if (index == 0) {
4         newNode.next = head;
5         head = newNode;
6         return;
7     }
8     Node current = head;
9     for (int i = 0; i < index - 1 && current != null; i++)
10        current = current.next;
11    if (current == null) return;
12    newNode.next = current.next;
13    current.next = newNode;
14 }

```

- الشرط الأول : عندما يكون $index=0$ أي نريد إضافة عنصر في بداية LinkedList كما ذكرنا سابقاً.
- تعريف current أيضاً من أجل الوصول إلى (الموضع) $index$ المطلوب؛ حيث تبدأ عند العقدة head وصولاً إلى الـ $index$ المحدد فإذا أصبحت current تشير إلى null، عندئذٍ الـ $index$ كبير غير موجود في الـ LinkedList و إلا فسيتم إضافة newNode في الموضع المحدد.

تذكرة :

return من أجل إيقاف الكود عند حالة من الحالات السابقة حتى لا يكمل البرنامج و يسبب أخطاء منطقية .

التعقيد: $O(n)$.

الدالة (4): حذف عنصر من البداية:

```
1 public void deleteFirst{
2     if(head == null)
3         return;
4     head = head.next;
5 }
```

- الشرط في البداية ليتحقق فيما إذا كان head يشير إلى null عندئذٍ تكون الـ LinkedList خالية.
- ثم نقطع العنصر الأول عن التسلسل بأن تصبح head تشير إلى العنصر الثاني.

التعقيد: $O(1)$.

الدالة (5): حذف عنصر من النهاية:

```
1 public void deleteLast() {
2     if (head == null)
3         return;
4     if (head.next == null) {
5         head = null;
6         return;
7     }
8     Node current = head;
9     while (current.next.next != null)
10         current = current.next;
11     current.next = null;
12 }
```

- إما أن تكون LinkedList خالية أي لا يفعل شيء return
- أو يكون فيها عنصر واحد فقط أي العقدة التي يشير إليها head تشير إلى null عندئذٍ head تؤشر على null و ننهي الربط بين node $\Rightarrow$ head .
- و في الحالة الأخيرة لدينا العقدة current و الهدف منها الوصول إلى العنصر قبل الأخير و جعله يشير إلى قيمة null.

التعقيد: $O(n)$.

الدالة (6): طباعة جميع العناصر:

```

1 public void printList() {
2     Node current = head;
3     System.out.print("LinkedList: ");
4     while (current != null) {
5         System.out.print(current.element + " -> ");
6         current = current.next;
7     }
8     System.out.println("null");
9 }

```

قمنا بتعريف عقدة current و أيضاً بدأت من head وصولاً إلى null.

■ التعقيد: $O(n)$

تابع لإيجاد منتصف الـ LinkedList:

بما أنه لا يوجد index في LinkedList سيكون من الصعب إيجاد منتصف LinkedList مباشرة. لدينا مؤشر فقط عند رأس LinkedList و أيضاً سنمرّ على العناصر وصولاً إلى المنتصف.

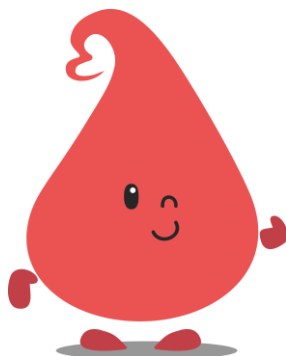
🔗 و لكن كيف يمكننا تحقيق ذلك؟

سنعرّف مؤشرين يبدأان من head، الأول يمشي خطوة واحدة في $slow = slow.next$ و الثاني يمشي خطوتين $fast = fast.next.next$ ، فعندما يصل الثاني fast إلى نهاية الـ LinkedList يكون الأول slow قد وصل إلى منتصف الـ LinkedList.

ملاحظة:

لا تخاف من الـ LinkedList لأنه الشغلة بسيطة كثير، بس ارسم العقد و حطّ الـ head و بلّش وصل المربعات (العقد) و هيك بتلاقي حالك كتبت الكود بشكل منطقي جداً.

سنقدم لكم المزيد من التمارين التي ستساعد في بناء طريقة التفكير الصحيحة



سنطوّر معاً على الكود التالي و نحوّل أن نقلّل التعقيد:

```
1 public static boolean isPrime(int n) {
2     if(n == 2 || n == 1) return true;
3     for(int i = 2; i < n; i++)
4         if(n % i == 0)
5             return false;
6     return true;
7 }
```

و الآن: نعلم أنّ الأعداد الزوجية تقبل القسمة على العدد 2 فهي إذاً غير أوليّة، و بهذا التعديل ستخفّض التعقيد من $O(n)$ إلى $O(n/2)$ ، لأننا سنمرّ على الأعداد الفردية فقط.

```
1 public static boolean isPrime(int n) {
2     if(n == 2 || n == 1)
3         return true;
4     if(n%2==0)
5         return false;
6     for(int i = 3; i < n; i++)
7         if(n % i == 0)
8             return false;
9     return true;
10 }
```

يكفي لمعرفة إذا كان العدد أولي هو أن نمرّ على الأعداد حتى جذره فمثلاً لمعرفة إذا كان العدد 16 هو عدد أولي، يجب أن نمرّ على الأعداد حتى جذره 4، فنجد أنّ العدد 16 يقبل القسمة على 4 فهو إذاً غير أولي و لا داعي للمرور على باقي الأعداد.

و بهذه الطريقة يمكن أن نختصر العمليات من $(n/2)$ عملية إلى $(\sqrt{n}/2)$ و هذا يوفر التعقيد و يضمن أنّ العمليات أقلّ، و نلاحظ أنّ هذا الفرق يظهر بوضوح عند الأعداد الكبيرة.

```
1 public static boolean isPrime(int n) {
2     if(n == 2 || n == 1)
3         return true;
4     if(n%2==0)
5         return false;
6     for(int i = 3; i*i <= n; i++)
7         if(n % i == 0)
8             return false;
9     return true;
10 }
```

و الآن أصدقاءنا دعونا نفكر سوياً في طريقة ترتيب مصفوفة ترتيباً تصاعدياً (من الأصغر إلى الأكبر):

من المؤكد سنبحث عن أصغر عدد في كل مرة وصولاً إلى مصفوفة مرتبة.

لكن كيف يمكننا كتابة هذه الخوارزمية على شكل كود ؟؟

سنحتاج حلقتي for، الأولى للمرور على جميع العناصر عنصر عنصر، و الثانية لكي نقارن و نبحت عن أصغر عنصر.

مثال:

5	4	2	1	6	3
---	---	---	---	---	---

- في أول مرة $i=0$
 $a[i]=5$
- في المقارنة الأولى العدد 4 أصغر من 5 سأحفظ الـ index الخاص بالعنصر حتى أستطيع القلب بين العنصرين (reverse) و ذلك في نهاية عملية البحث عن العنصر الأصغر.
- نكمل عملية المقارنة ($4 > 2$) نحفظ الـ index الجديد بما أن العدد أصغر.
- مجدداً ($1 < 2$) نحفظ الـ index الجديد .
- العدد ($6 > 1$) و نكمل.....
- العدد ($3 > 1$) و هكذا نكون قد مررنا على جميع العناصر و أصغر index لدينا هو $index=3$ ، نقلب بين
 $reverse(a[0], a[3]);$

الآن أصبحت المصفوفة بالشكل :

1	4	2	5	6	3
---	---	---	---	---	---

و الآن مع العنصر التالي :

- $i=1$
 $a[i]=4$
- نكرر نفس عملية المقارنة السابقة، حيث سيكون أصغر عدد عند $index$ هو $index=2$ ، بعد المرور على جميع العناصر ; ($a[1], a[2]$)
 $reverse(a[1], a[2]);$

1	2	4	5	6	3
---	---	---	---	---	---

و بتكرار هذه العملية نحصل على مصفوفة مرتبة.

التعقيد: $O(n^2)$.

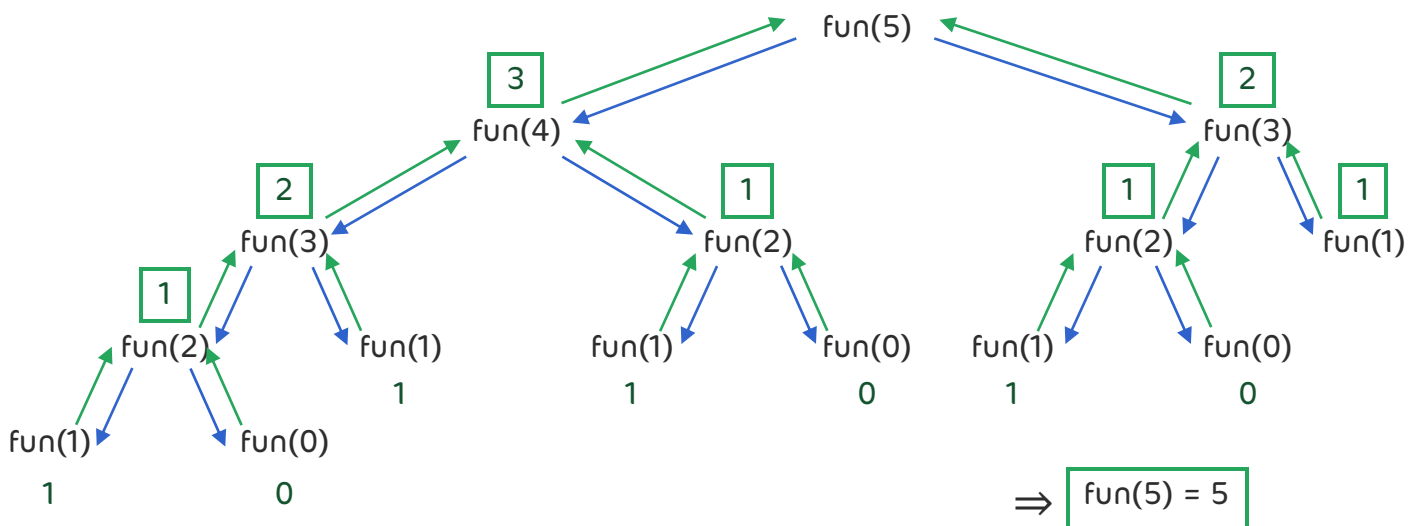
```

1  int[] a = {3, 2, 4, 8, 12, 5};
2  int n = a.length;
3  for (int i = 0; i < n; ++i) {
4      int mn = i;
5      for (int j = i + 1; j < n; ++j) {
6          if (a[mn] > a[j]) {
7              mn = j;
8          }
9      }
10     if (mn != i) {
11         int temp = a[i];
12         a[i] = a[mn];
13         a[mn] = temp;
14     }
15 }

```

تسمّى هذه الخوارزمية selection sort، تعقيدها المكانيّ ممتاز، كل العمليّات تتمّ (in_place) في نفس المكان، دون الحاجة لمساحة إضافية، وستكون خيار جيد إذا كانت المصفوفة صغيرة (n صغير)، و لكنّ التعقيد الزمنيّ سيّء، إذاً هناك حلول أفضل وسنأخذها لاحقاً لذا لا تقلقوا يا أصدقاء.

حل تمرین فیوناتشی:



التعقيد للتابع العودي هو $(Branches^{depth})$ ، أي عمق مستويات الشجرة (عدد الفروع).

نلاحظ في التابع السابق أن كل استدعاء ينتج عنه فرعين (بشكل مضاعف للعدد 2).

⇐ يوجد فرعين و عمق الاستدعاء هو بحسب الدخل $O(2^n)$.

فوائد:

1. لاحظنا أن الفروع يتكرر حسابها مما يزيد التعقيد الزمني، لكن يوجد طريقة للتقليل من هذه الاستدعاءات المتكررة و تدعى البرمجة الديناميكية dp.
2. عند رسم الشجرة ابدأ بفرع و أكمله؛ و أرجع قيمته للذي قبله (فوقه) و من ثمّ ابدأ بالفرع الثاني و أرجع قيمته، و هكذا...

لا تقبل بهدف أقل من القمة مهما كلفك الأمر...

فأنت تستطيع ذلك وتستحق ذلك

يقول المتنبي :

"إذا غامرتَ في شرفٍ مرومٍ

فلا تقنع بما دون النجومِ

فطعمُ الموتِ في أمرٍ حقيرٍ

كطعمِ الموتِ في أمرٍ عظيمٍ"

